

METRIC INTERFACE · 这一周到底要立什么

1. 凭什么说 Week5 不是"做几个指标"

指标不是 SQL 的结果，是要能被多方一起负责的工程接口。

这一节先把"指标 = SQL"这个直觉打掉——这是我做了十几年大数据，事故复盘出来的第一个判断。

本课交付

- 一份"指标接口卡"（先用纸写，5 分钟出草稿）
- 一份 ADR：为什么我们走"本地 dbt + registry"，而不是 dbt Cloud / Cube
- 把判断翻译成 OmniSupport 的两份蓝图：`adr-week5-analytics-path.md` / `metric-interface-principles.md`

[开场判断](#)[事故视角](#)[工程入口](#)[行业信号](#)[Recap/Next](#)

Week05 学习链

01

Metrics

把业务口径变成工程接口

02

dbt Layers

sources / staging / intermediate / marts 的消费边界

03

Evidence

tests / docs / lineage 证明口径能负责

04

Semantics

MetricFlow、Semantic Models 与本地 registry 边界

05

Tool Guard

不让 Agent 裸写 SQL：受控指标查询工具

WAR STORY · 我吃过的一次"指标"亏

2020 年某电商：CFO 一句话，把我钉在凳子上三周

这件事，是我后来到处推广"指标接口"这个判断的起点。

事故现场

CFO 月会前一周问运营：「P1 工单这季度到底涨了多少？」三个团队，三张报表。

运营看板说 +28%；BI 临时拉一遍说 +41%；客服总监手里 PM 给的 PPT 说 +12%。差距整整 29 个百分点。三方都不肯让。CFO 当场拍板：「下次开会，我只看一个数。」

查到最后的根因（没有一个是"SQL 写错"）

① 运营按 created_at 算"新建 P1"；② BI 按 last_status_change_at 把历史升级也算进来；③ 客服那张是按 priority_changed_at 加了 SLA tier 过滤。三个 SQL 都跑得通，三个口径都"合理"，没有一个能在公司内部被多方共识。

我后来花了三周做了一件事：把这个组织里所有"和 P1 工单"相关的口径，重做成一份「指标接口」——有 owner、有 grain、有时间字段定义、有谁能查、有审计。从那以后 CFO 月会再没出过同样的事。这是我"指标 = 工程接口"判断的源头。

ROLE MAP · 给你做个工种地图

是"分析工程师 + AI 工程师"两顶帽子

搞清你和隔壁两个工种的边界，后面所有讨论才有意义——别越权干活，也别等着别人干。

Data Engineer

上一周（Week01-04）你戴的帽子。

职责：把数据从源系统、日志、第三方搬进仓里，保证可信。

产物：管道、Bronze/Silver 表、SLA。

Analytics Engineer

本周你的主帽子。这是 2020 年随 dbt 火起来的新工种。

职责：把仓里的事实，做成业务能消费的口径。

产物：dbt models、指标卡、registry、文档、测试。

AI / Agent Engineer

本周后半场你会切到这顶帽子。

职责：把指标交给 LLM，同时不让它越权。

产物：tool contract、安全视图、审计、HITL。

Data Analyst / BI

夕阳工种吗？不是。但 LLM 时代他们的角色在变。

以前他们是"取数"的；以后他们是"定义口径 + 解读结果"的。

ML / Data Scientist

Week05 你做的 mart，未来是他们特征库的合法上游。

所以你做得好不好，他们立刻有感觉。

Governance / Security

以前他们等事故来；现在你给他们一个工程层的入口。

Week05 的 audit_id / release_id 就是他们的"安全带"。

WHY NOW · 为什么这事 2026 突然必修

LLM 把"同名不同义"从治理问题升级成线上事故

这事十年前我也讲，但当年只是"开会吵"。现在多了一个会自信地把错口径写成 SQL 的家伙。

时代	同名不同义的代价	工程师该不该立马修
2010-2018	运营开会吵半小时，最后用 Excel 调和	不该；ROI 太低
2019-2023	看板互相打架，CFO 上 BI 中台，定义"指标平台"	该，但还不紧急
2024-2026	Agent 用错口径"自信地"答了客户、写了报告、决策了退款	必修。一次 PR 事故的赔偿，够你做三年指标治理

三个行业信号，证明你不是在跟风

- ① 2025 Coalesce：dbt Labs 把 MetricFlow 改成 Apache 2.0。「治理过的指标」第一次开源进入所有团队。
- ② 2026.01：OSI（Open Semantic Interchange）v1.0 发布。dbt Labs、Cube、Atlan、Snowflake 都签了——你不需要押注任何一家。
- ③ dbt Labs 实测：直连 LLM 写 SQL，准确率惨；走语义层，可回答的问题里 83% 正确。这 40 多个百分点的差距，就是 Week05 的工程价值。

KEY THESIS · 我反复给团队讲的判断

没有 grain / owner / source / filters / tests / audit / Docs, 它不叫接口

你公司里 80% 写在 wiki 上叫"指标定义"的东西, 按这个标准其实都不及格。

BI 的诉求

稳定显示, 不要每张看板复制一份口径。
他们最痛的是"上面问问题, 三张图三个数"。

Agent 的诉求

受控查询, 别让我"自由发挥"。
LLM 的特点是"高自信地错"。你不给它边界, 它就给你制造事故。

治理的诉求

可复盘可追责。出事的时候, 能说清"哪天的数、哪一版语义、谁查的"。
没有这三件, 治理团队连签字都不敢。

你下次新加一个指标前, 先停三分钟, 把这 7 件事先答完, 再动 SQL。这是最便宜的"事故止损器"。

BAD CASE · 不是 SQL 错，是定义没立

同一个问题："最近 7 天 P1 工单是不是涨了？"

这种"四方各算各的"，是我在企业里见得最多的事故剖面。

谁回答	怎么算	数字差异来自哪	暴露的真问题
运营看板	按 created_at 聚合新建 P1	时间字段无约定	没人写明"P1 用哪个时间"
BI 临时报表	按 updated_at + 历史升级算	grain / filters 无约定	没有 owner，每个分析师拼一遍
Agent 工具	查底表 / 自己生成 SQL，可能带出 PII	工具边界和审计无约定	没有 registry，也没 safe view
评测复盘	复用某张 dashboard 截图	数据 release_id 无约定	没有可复现的事实快照

四方都"能算"，但没人能"负责"——Week05 修的就是这条裂缝，不是 SQL 写法。

CONCEPT SPLIT · 把指标拆成三个工程对象

metric / business definition / engineering interface

中文里都翻译成"指标", 但生产里它们是三个不同的对象。混了, 就栽。

对象	答什么问题	生产例子	混了的后果
metric · 指标	业务上要复用、要解释的那个数字	p1_ticket_count	同名不同义; 月会上吵半天
business definition · 业务口径	怎么算、什么粒度、什么时间窗、reopen 算不算	P1 = priority in (p1, p1_critical) 且按 created_date	SQL 跑得通, 但口径漂; 事故先甩锅
engineering interface · 工程接口	谁能查、怎么查、输出什么、如何审计	query_support_kpis_v1 + registry + audit_id	Agent 越界、无法复盘、不可评测

metric 是名字, definition 是含义, interface 让多方对得齐。

METRIC CARD · 我十几年用下来的最低必填集

16 个字段，没有一个是装饰——少一个，事故就以那个字段的形状出现

这张卡你可以扩，但不能减。扩出来的字段是"团队偏好"，删掉的是"事故种子"。

字段	说明	p1_ticket_count 示例	字段	说明	p1_ticket_count 示例
metric_name	稳定机器名	p1_ticket_count	allowed_dimensions	允许维度	product_line / priority / org_id / category
business_label	人能读的名字	P1 工单数	allowed_filters	允许过滤	同上 + data_release_id
business_question	它在回答的问题	最近 P1 是否变多 / 集中在哪些产品线	owner	口径负责人	support_ops
business_definition	业务定义	窗口内创建的 P1 或 P1_critical 工单数	tests	必要测试	not_null / accepted_values / registry validation
source_model	来源 mart	support_kpi_mart	docs_link	文档入口	dbt docs · metric-interface-principles.md
grain	一行代表什么	metric_date × product_line × priority × org_id × category	allowed_roles	允许角色	support_ops / instructor / admin
measure_expression	计算逻辑	sum(p1_ticket_count)	audit_fields	审计字段	actor_role / actor_id / release_id / row_count
time_field	时间字段	metric_date	change_policy	改口径要做什么	先写 impact note，再更新 tests / docs / registry

CHECKLIST · 上线前的 10 个问题

答不上来一项 = HOLD。HOLD 不是失败，是工程纪律

这 10 条我在大厂和外企做架构评审时都在用。任何一条没答出来，这个指标当周不上线。

01 Owner

谁负责解释和修复？

02 Grain

一行到底代表什么？

03 Time field

created_at / updated_at /
event_time?

04 Filters

默认过滤条件是什么？

05 Dimensions

开放哪些；不开哪些？

06 Tests

哪些断言保护口径？

07 Docs

消费者读得懂吗？

08 Lineage

变更影响能定位吗？

09 Roles

哪些角色能查？

10 Agent gate

能通过受控工具调用吗？

Hold 一个指标，比修一次事故便宜十倍。

COMMON MISREADS · 你大概率信过其中两条

这 4 句话我在客户现场反复听到——每句话背后都至少一次事故

我做架构评审第一件事，就是看团队是不是还在这四个误区里。

你大概率信过	现实是	它会让你赔的代价
"指标就是 SQL"	指标是定义 + 来源 + 粒度 + 测试 + 文档 + 权限 + 接口的合体	月会三个数；CFO 拍桌
"Dashboard 显示了就够"	Agent / Eval / 治理 / 运营复盘都要同一口径	AI 答错后没人能复盘
"SQL 改了大家同步一下"	没有 lineage 和 docs，影响范围根本不可控	旧口径继续被消费 6 个月没人发现
"Agent 会自己写对 SQL"	它会"高自信地"写出能跑、口径错、越权、不可审计的 SQL	一次 PR 事故 ≈ 你三年指标治理预算

LANDING · 看看 OmniSupport 怎么落这套

这套方法论，到我们项目里就长成 11 个指标 + 1 份 registry + 1 个工具

项目是给你一个"能动手摸"的落地示范——你回去做的任何客服 / 销售 / 风控的指标包，结构都是一样的。

指标	业务问题	粒度	最危险误差	主要消费者
p1_ticket_count	P1 是否变多	daily metric	时间字段选错	BI / Ops / Agent
sla_breach_count / rate	SLA 是否失守	daily metric	状态窗口未约定	Ops / Eval
open_ticket_count	当前积压规模	daily metric	closed 口径错	Ops / Agent
escalation_count / rate	升级是否变多	daily metric	重复计数	Ops / Agent
avg_first_response_minutes	首响是否变慢	case-level avg	忽略业务时区	BI / SLA
first_resolution_rate △ experimental	一次解决率（代理）	ratio	分母混乱；reopen 未接入	Eval / Ops

△ first_resolution_rate 被我故意留成 experimental_proxy——它示范一件事：你做指标可以"诚实地"标"代理口径"，比偷偷上线高级一万倍。

EXERCISE · 现场练习

5 分钟动手：照模板，把 sla_breach_rate 写一张卡

不写 SQL，写卡。卡写好了，SQL 是水到渠成的事——这是真实的"分析工程师"日常。

Step 1

metric_name + owner
+ business_question



Step 2

grain + source_model
+ time_field



Step 3

dimensions + filters
+ allowed_roles
+ max_window_days



Step 4

tests + docs_ref
+ audit_fields
+ downstream

```
metric_name: sla_breach_rate
business_label: SLA 违约率
business_question: 服务承诺是不是在被打破?
business_definition: SLA 违约工单数 / 工单总数
source_model: support_kpi_mart
grain: [metric_date, product_line, priority, org_id, category]
measure_expression: avg(sla_breach_rate) # 已在 mart 里聚合好
time_field: metric_date
allowed_dimensions: [product_line, priority, org_id, category]
allowed_roles: [support_ops, instructor, admin]
max_window_days: 31
tests: [between_0_and_1, denominator_not_zero_when_reported]
docs_link: docs/blueprints/week05/metric-interface-principles.md
audit_fields: [actor_role, actor_id, release_id, row_count]
change_policy: ratio 改口径必须双写一段时间，并发邮件给 BI/Ops
```

YAML ·

```
metric_card /
sla_breach_ra
te
```

SELF-CHECK · 这节课你过没过

离开这节课前，问自己 6 个问题

答得出 = 这节课可以收尾。答不出 = 这节课你只听了，没听进去。

① 我能用一句话解释为什么 KPI 不是 SQL 结果

② 我能说清 BI / Agent / Eval / 治理为什么必须共享一套口径

③ 我能指出 P1 工单事故里三方数字不同的根因

④ 我能列出 Agent 裸写 SQL 至少三类风险

⑤ 我能为任意一个指标写出 grain / source / time field / dimensions

⑥ 我能解释为什么 first_resolution_rate 是 experimental_proxy

课后 30 分钟最小行动（这条做完，下节课跟得动）

在你的 notes 里抄一张空白「指标接口卡」16 字段，把它分别填完 p1_ticket_count、sla_breach_rate、open_ticket_count 三个指标。

填完之后看一眼：哪两三个字段你卡住了？把那几个字段名圈出来——它们就是你这一周最该补的工程能力。

RECAP & NEXT

这节课你应该带走的 7 个判断

本课最重要的判断

- 指标不是 SQL，是 metric 名字 × business definition × engineering interface 的合体。
- grain（一行是什么）比 SQL 写法重要十倍——grain 错了，看着都对，全是错。
- owner / source / filters / dimensions / max_window 五个字段必须可见，缺一就是事故种子。
- BI、Agent、Eval、治理必须共享同一口径——不是想不想，是 2026 的合规底线。
- 指标接口卡 16 字段是"最低集合"，不是装饰；少一个，事故就以那个字段的形状出现。
- 上线前 10 个问题，任一答不出 = HOLD。HOLD 比事故便宜十倍。
- 下一讲：用 dbt 分层，让指标接口从 source 自然长出来。

下一步

Lesson 02 会逐层解剖 sources / staging / intermediate / marts，并把"为什么这样切"讲透——这是我做架构这么多年最被低估的一节课。

你将带走：「分层 = 消费边界 + 污染隔离」的工程直觉，不再背口诀。

DBT LAYERS · 这节课讲一件事

2. dbt 分层不是教条

分层不是目录命名，是「上游变化不污染下游」的工程纪律。

我做架构这么多年最被低估的一节——背口诀的人很多，真正懂为什么的不到20%。

本课交付

- 一份"为什么 4 层"的决策记录（不是默认照抄）
- 能讲清 staging / intermediate / marts 各自的"不该做的事"
- 在 OmniSupport 项目里跑通 `dbt build --select tag:week05`

开场判断

事故视角

工程入口

行业信号

Recap/Next

Week05 学习链

01

Metrics

把业务口径变成工程接口

02

dbt Layers

sources / staging / intermediate / marts 的消费边界

03

Evidence

tests / docs / lineage 证明口径能负责

04

Semantics

MetricFlow、Semantic Models 与本地 registry 边界

05

Tool Guard

不让 Agent 裸写 SQL：受控指标查询工具

WAR STORY · 没 dbt 的年代我们怎么死

2018 某物流公司：一次上游枚举改名，下游 14 处同时挂掉

你回头看，dbt 火不是偶然——它在解决一类"散落 SQL"的工程灾难。

事故现场

上游同事把 status 从 `"In Transit"`（字符串）改成枚举 `IN_TRANSIT`（大写）。他在自己的 service 里 PR 全绿。

下游 14 处同时挂——3 张 BI 看板、4 个 cron 跑的报表 SQL、2 个 stored procedure、2 个 Python 数据脚本、2 个 Hive 视图、1 个客户对账接口。

为什么这种事会发生

① 没有 sources 声明：上游表谁在用，没人能查；② 没有依赖图：改动影响范围靠"我大概记得"；③ 没有测试：枚举漂移没人发现；④ 修复全靠 grep。我们事故复盘那天，CTO 一句话："这事必须靠工具，不靠人记。"

后来 dbt 第一次让 SQL 拥有"代码的属性"：依赖图、测试、文档、可复现命令。这节课的所有分层、所有 mart、所有 view，本质上都在防那次 14 处挂掉的故事。

HISTORY · 数据栈过去 15 年

ETL → ELT → Analytics Engineering: dbt 出现的原因

不讲清楚这条路径，你就把 dbt 当工具用；讲清楚，你才会把 dbt 当方法论用。

时代	主流路径	埋下的雷	代表工具
2005-2015 · ETL	源系统 → ETL 服务 → 仓（变形在仓外）	Python/Spark 脚本不进版本控制；分析师够不着；改一次跑半小时	Informatica / Talend / 手写 PySpark
2015-2020 · ELT	源系统 → 仓内 staging → 仓内 transform（仓外只搬不变）	仓内 SQL 散落，没依赖、没测试、没文档；新人一脸懵	Fivetran / Airbyte + 仓内手写 SQL
2020+ · Analytics Engineering	SQL 工程化：模型 + 依赖 + 测试 + 文档 + DAG 编译	新工种诞生：分析工程师。指标第一次能"被工程"	dbt Core / dbt Cloud / SQLMesh

dbt 把"写 SQL"变成了"写代码"——它逼你像写程序一样写指标。这是它真正的工程价值。

FUNDAMENTALS · 把 dbt 拆给零基础同学

dbt 工程的 6 块基本盘

这一页你看明白，整个 dbt 体系就立住一大半——剩下的都是细节。

① 模型 = SQL 文件

每个 .sql = 一个表 / 视图。
dbt 编译成 CREATE TABLE / VIEW 落仓。
dbt 不存数据，它只生成 SQL。

② Jinja 模板

SQL 里写 `{{ ref(...) }}` / `{{ source(...) }}` / `{{ var(...) }}`。
SQL 第一次拥有变量、循环、宏。

③ 依赖 DAG

只要写 `{{ ref("stg_tickets") }}`，dbt 就知道依赖。
拓扑排序自动算运行顺序。

④ Tests

内置 `not_null` / `unique` / `accepted_values` / `relationships`，
再加自定义 SQL test。每次 build 自动跑。

⑤ Docs + Artifacts

``dbt docs generate`` → `manifest` / `catalog` / `run_results`。
血缘网站 + 审计证据，两件事一次到位。

⑥ Materialization

`view` / `table` / `incremental` / `ephemeral`。
这是“成本 × 新鲜度”决策点；不是越大表越 incremental。

dbt 的核心价值：让 SQL 写法“具备工程对象的所有属性”。

IS / IS NOT · 别把 dbt 用偏

dbt 是 SQL 工程化框架，不是数据库、不是 BI、不是采集

我评审过的 30 多个团队，至少一半在这五件事上误用——结果就是 dbt 装了，但没拿到它的红利。

你可能以为	其实不是	它真正负责的
dbt 是数据库	它不存数据；数据在仓里	SQL transform 的工程框架
dbt 是 BI	它不画图、不交互	把模型和口径变成可复现对象
dbt 是采集工具	它不负责 EL；上游要先有数据	承接已加载的数据，再加工
dbt 是 SQL 文件夹	只堆 SQL 不叫 dbt	依赖、测试、文档、DAG、artifacts
dbt = 语义层	dbt 本身不暴露指标 API（Semantic Layer 是另一层）	语义层定义的事实来源

KEY THESIS · 分层到底为了啥

分层 = 污染隔离 + 消费边界 + 影响可定位

不是给目录起好听名字，是给"变化"画边界。

污染隔离

上游漂了，控制在 staging。
业务规则变了，控制在 intermediate。
消费方变了，只动 marts 不动底层。

消费边界

每一层有"该做的"和"绝对不该做的"。
mart 的字段不是"塞越多越好"，是"刚好够下游用"。

影响可定位

依赖图清晰，"改了 X 谁会受影响"一分钟看得到。
这就是开头那个 14 处挂掉事故的解药。

判断一个团队的 dbt 工程好不好，不看模型数量，看"目录之间该做的事是不是泾渭分明"。

LAYER 1 · SOURCES

承认真实上游——这一层越简单越稳

很多人在 sources 里写 cast、写 case-when，全错。这一层只做一件事：让所有下游模型有同一个事实来源。

```
# analytics/models/sources.yml (OmniSupport 项目示范)
sources:
  - name: omni_postgres
    description: "Week03/Week04 已经加载到 PostgreSQL 的结构化工单数据"
    database: "{{ env_var('POSTGRES_DB', 'omnisupport') }}"
    schema: public
    tables:
      - name: ticket_fact
        columns:
          - name: ticket_id
            tests: [not_null, unique]
          - name: created_at
            tests: [not_null]
      - name: customer_dim
      - name: ticket_comment_fact
      - name: knowledge_doc
```

sources.yml

可以做

声明物理上游表。

挂关键字段约束测试（not_null / unique）。

配 freshness（多久新鲜一次）。

不要做

不做 cast、不做 rename（rename 是 staging）。

不写 case-when。

不让 dashboard 直接引用。

为什么这层这么节制

sources 是上游和下游之间的契约。

越简单越稳；一旦复杂，上游一变这层就崩。

LAYER 2 · STAGING

整形输入，绝不塞 KPI

我见过的最常见反模式：staging 里塞了 group by。这是把"事故种子"种在最深的那一层。

```
-- analytics/models/staging/stg_tickets.sql
with source as (
  select * from {{ source('omni_postgres', 'ticket_fact') }}
),
renamed as (
  select
    ticket_id, customer_id, org_id,
    lower(status::text) as status,      -- 枚举标准化
    lower(priority::text) as priority,
    cast(created_at as date) as created_date, -- 派生稳定日期字段
    coalesce(updated_at, created_at) as updated_at,
    case when lower(status::text) in ('open','pending_customer','in_progress','escalated')
      then true else false end as is_open,
    case when lower(priority::text) in ('p1_critical','p1')
      then true else false end as is_p1,
    case when sla_due_at is null then false
      when coalesce(resolved_at, current_timestamp) > sla_due_at then true
      else false end as sla_breached
  from source
)
select * from renamed
```

stg_tickets.sql

可以做

rename / cast / 枚举标准化。
派生纯类型布尔（is_open / is_p1）。
PII 标记，不删数据加标志。

坚决不要

不做 group by 聚合。
不写 dashboard 专用字段。
不做跨表 JOIN 业务逻辑（那是 intermediate）。

"派生布尔 OK，聚合 KPI 不 OK"——这是 staging 与 intermediate / marts 的最重要分界。

LAYER 3 · INTERMEDIATE

业务组合的"集中入口"，不直接面向消费者

这是我个人最爱的一层。它的存在让"业务逻辑改一次就行"，而不是"改十处都改对"。

```
-- analytics/models/intermediate/int_support_cases.sql
with tickets as ( select * from {{ ref('stg_tickets') }} ),
    customers as ( select * from {{ ref('stg_customers') }} ),
    first_comments as (
        select ticket_id, min(created_at) as first_comment_at
        from {{ ref('stg_ticket_comments') }}
        group by ticket_id
    ),
    joined as (
        select t.ticket_id, t.org_id, c.org_name,
            t.status, t.priority, t.created_date, t.resolved_at,
            fc.first_comment_at,
            case when fc.first_comment_at is not null
                then extract(epoch from (fc.first_comment_at - t.created_at)) / 60.0
                else null end as first_response_minutes, -- 派生业务事实
            case when t.resolved_at is not null
                then extract(epoch from (t.resolved_at - t.created_at)) / 86400.0
                else extract(epoch from (current_timestamp - t.created_at)) / 86400.0
                end as backlog_age_days,
            case when t.resolved_at is not null and not t.is_escalated then true
                else false end as is_first_resolution_proxy
        from tickets t
        left join customers c on t.customer_id = c.customer_id
        left join first_comments fc on t.ticket_id = fc.ticket_id
    )
select * from joined
```

int_support_c
ases.sql

这一层做的事

JOIN ticket + customer + comment。
派生"业务事实"：first_response_minutes、
backlog_age_days、is_first_resolution_proxy。
一行还是 case 粒度。

边界

不直接给 BI / Agent 消费——不稳定。
不开放给外部团队引用——是内部细节。
不做最终聚合（那是 mart）。

LAYER 4 · MARTS

marts 不是"最后一个文件夹", 是"对外契约"

评审一个 mart 是不是合格, 我有条铁律: 能不能用一句话说清「一行 + 谁消费 + 不能暴露什么」。

support_case_mart

grain = 一个 support case。
供分析师 / BI 做 case-level 切片。
字段: "安全的、有业务含义的"事实。

support_kpi_mart

grain = 指标 × 日期 × 维度。
11 个 metric 全走它。
Week05 的"指标公共边界"——本周最核心 mart。

agent_tool_input_view

物化是 view。
只暴露白名单字段。
Agent 唯一能读的对象——把 PII / 正文物理性地排除。

一句话判断

打开 schema.yml, 说不出"一行是什么 + 谁消费 + 不能暴露什么"——这个 mart 还不该被消费。

GRAIN · 工程师之间最便宜的共识

grain 错一次，看着都对，结果全错

grain 是我"最被低估的工程概念"评分第一。它一句话出来，整个团队对得齐。

grain	一行是什么	错了会怎样（我都见过）
case grain	一个 support case	复开率分母混乱；reopen 被重复算
daily activity grain	某天 × 某类活动 × 某维度组合	历史升级被重复算；漏掉时区
metric row grain	某指标 × 某日期 × 某维度组合	不同指标被混到同一行；列错位
query result grain	工具返回的结构化结果	Agent 汇总错维度；下游图表说谎
snapshot grain	某时刻的当前状态快照	把"已关闭"算成"当前打开"

我评审 mart 的第一句话永远是："这一行代表什么？"——答不上来，PR 不批。

LANDING · support_case_mart 解剖

看 OmniSupport 怎么把"事实边界"做出来

重点不在 SQL 本身，在于：什么字段被选了 / 什么字段被故意没选。

```
-- 选出来的：安全且有业务含义的字段
select
  ticket_id, org_id, org_name, status, priority,
  category, product_line, product_version, assignee_id,
  sla_tier, sla_due_at,
  created_at, created_date, updated_at, resolved_at,
  first_comment_at,
  first_response_minutes,    -- 来自 int_support_cases
  handle_time_minutes,
  backlog_age_days,
  is_open, is_resolved, is_escalated, is_p1, sla_breached,
  is_first_resolution_proxy,
  pii_level, pii_redacted,   -- 治理标志，不是 PII 内容
  data_release_id,          -- 审计与复盘
  ingest_batch_id, schema_version
from {{ ref('int_support_cases') }}
```

```
-- 故意没选的：customer_email / phone / subject 全文 / comment body
-- 一行不放，全部下游都不可能拿到。这就是"工程纪律"。
```

support_case
_mart.sql

它保护的指标

reopen_rate（未来接入）
active_ticket_count
sla_breach_count
backlog_age 分布

严禁暴露的

customer_email、phone
工单 subject 全文
comment body 原文
IP / 设备指纹

留 pii_redacted 的细节

这是治理标志（脱敏过），不是 PII 本身。保留它让下游知道"这条记录是脱敏的"。

LANDING · support_kpi_mart 解剖

这段 SQL 我希望你看 3 遍——它是"BI/Agent/Eval 共享口径"的工程关键

宽表→长表的翻转技巧，是把 11 个指标全部固化进同一张表的核心手法。

```
with daily as (  
  select activity_date as metric_date, product_line, priority, org_id, category,  
         ticket_count, open_ticket_count, p1_ticket_count, sla_breach_count,  
         escalation_count, resolved_ticket_count, first_resolution_count,  
         avg_backlog_age_days, avg_first_response_minutes, avg_handle_time_minutes,  
         escalation_rate, sla_breach_rate, first_resolution_rate  
  from {{ ref('int_ticket_activity_daily') }}  
)  
metric_rows as (  
  -- 关键技巧: PostgreSQL 的 cross join lateral values  
  -- 把"宽表"翻转成 metric_name + metric_value 的长表  
  select daily.metric_date, metrics.metric_name,  
         daily.product_line, daily.priority, daily.org_id, daily.category,  
         metrics.metric_value  
  from daily  
  cross join lateral (  
    values  
    ('ticket_count', daily.ticket_count::numeric),  
    ('p1_ticket_count', daily.p1_ticket_count::numeric),  
    ('sla_breach_count', daily.sla_breach_count::numeric),  
    ('avg_first_response_minutes', daily.avg_first_response_minutes::numeric),  
    ('escalation_rate', daily.escalation_rate::numeric)  
    -- ... 11 个指标全部一次性翻转  
  ) as metrics(metric_name, metric_value)  
  where metrics.metric_value is not null  
)  
select metric_date, metric_name, product_line, priority, org_id, category,  
       metric_value,  
       '{{ var("week05_data_release_id") }}' as data_release_id,  
       current_timestamp as generated_at  
from metric_rows
```

support_kpi_
mart.sql

你以后做客服 / 销售 / 风控指标包，长表骨架都是这套——记住它。

SAFE VIEW + MATERIALIZATION

安全视图为什么是 view；物化策略到底怎么选

两件常被混在一起讲又被讲不透的事，我一次说清。

```
agent_tool_in
put_view.sql

{{ config(materialized='view') }}
select metric_date, metric_name, product_line, priority, org_id, category,
       metric_value, data_release_id, generated_at
from {{ ref('support_kpi_mart') }}
where metric_name in (
  'ticket_count','open_ticket_count','p1_ticket_count',
  'sla_breach_count','escalation_count',
  'avg_backlog_age_days','avg_first_response_minutes',
  'avg_handle_time_minutes',
  'first_resolution_rate','escalation_rate','sla_breach_rate'
)
-- 物理上"不存在"的字段: ticket_id / customer_id / subject / body / phone
```

物化	什么时候用	本项目选哪个
view	中小数据；要"最新"	staging / intermediate / safe view
table	稳定列、稳定快照	support_case_mart / support_kpi_mart
incremental	大表，按日期/批次增量	本周不用；事件流大表才需要
ephemeral	只被 1-2 个模型用的中间步骤	本周不用；保留作为优化项

safe view 为什么必须是 view：要实时跟上 mart 的最新构建；做成 table 会有"两份数据"的隐患。

物化口诀：staging/intermediate 默认 view（便宜）；marts 默认 table（稳定）；大表才考虑 incremental。

RUN · 跑通三条命令 + 自检

不背命令，看三件事：connection / models / artifacts

录播时这一页对着念命令很无聊；真正要让学员带走的是"该看哪几个证据"。

1) 启动依赖

```
docker compose -f infra/docker-compose.yml up -d --build postgres minio minio_init
```

2) dbt debug —— 看到 "Connection test: OK connection ok" 才往下

```
docker compose --profile tools run --rm devbox \
  bash -lc 'cd analytics && DBT_PROFILES_DIR=. dbt debug'
```

3) build + docs —— 一次出齐 manifest / catalog / run_results

```
docker compose --profile tools run --rm devbox \
  bash -lc 'cd analytics && DBT_PROFILES_DIR=. dbt build --select tag:week05'
```

```
docker compose --profile tools run --rm devbox \
```

```
  bash -lc 'cd analytics && DBT_PROFILES_DIR=. dbt docs generate'
```

bash

① 我能讲清四层各自的"不该做的事"

② 我能用一句话说出 case_mart / kpi_mart / safe_view 的 grain

③ 我没把 KPI 聚合写进 staging

④ 我能解释 LATERAL VALUES 长表翻转的工程意义

⑤ 我知道 safe view 为什么必须是 view

⑥ 我知道大表才考虑 incremental，不是越大越用

RECAP & NEXT

这节课你应该带走的 7 个判断

本课最重要的判断

- dbt 不是数据库 / BI / 采集——它是 SQL 工程化框架。
- 分层 = 污染隔离 + 消费边界 + 影响可定位；不是目录命名。
- sources 越简单越稳；staging 整形不塞 KPI；intermediate 是业务组合的集中入口。
- marts 不是"最后文件夹"，是对外契约；说不出 grain 就不该被消费。
- metric-row 长表（CROSS JOIN LATERAL VALUES）是 BI/Agent/Eval 共享口径的工程关键。
- safe view 必须是 view；物化按"新鲜度 × 成本 × 下游"挑，不是越大越 incremental。
- 下一讲：把"能跑的 SQL"升级成"能负责的口径"——证据链。

下一步

Lesson 03 会把工程纪律讲在前、dbt 工具落在后。
你将带走："能跑 ≠ 可交付"的工程纪律——tests / docs / lineage / artifacts 全部要齐。

EVIDENCE · 工程纪律先于工具

3. 能跑 ≠ 能交付

SQL 改对了，系统也能答错——这是我评审过 80% 团队的真实剖面。

这节课讲一件事：让指标能"被人和 Agent 共同信任"，要的不是更炫的 SQL，是证据链。

本课交付

- 一份 `schema.yml` (不是占位描述，是写给消费者的"接口说明")
- 一份自定义 PII 测试 (`tests/no\_pii\_columns\_\*.sql`)
- 一份 `dbt\_build\_evidence.md` (可被 reviewer 在 3 分钟内复核)
- 一份 `lineage-impact-notes.md` (变更影响清单)

开场判断

事故视角

工程入口

行业信号

Recap/Next

Week05 学习链

01

Metrics

把业务口径变成工程接口

02

dbt Layers

sources / staging / intermediate / marts 的消费边界

03

Evidence

tests / docs / lineage 证明口径能负责

04

Semantics

MetricFlow、Semantic Models 与本地 registry 边界

05

Tool Guard

不让 Agent 裸写 SQL: 受控指标查询工具

WAR STORY · "reopen 算不算" 的半年公案

某物流公司：一个看似简单的字段含义，吵了半年没人能签字

这是真实事故——也是我后来给所有团队推"证据链 = 交付门槛"的源头。

事故现场

客户支持团队的"工单数"指标，运营说 +18%，产品说 +6%。复盘发现：差异全在「已关闭后重新打开（reopen）」算不算一次新工单。

我进去做架构那天，问了三个问题——这个口径的 owner 是谁？docs 写在哪？变更过哪几次？三个问题都没人能答。SQL 是 2017 年的某前同事写的，文档没有，他人离职了。

真正坏点

不是 SQL 写错——SQL 一直在跑。是没人能解释、没人能追责、没人能复盘。这就是"有指标无证据链"的代价：每次出现争议，整个组织停摆一周。

当时我给团队的判断：能跑的 mart 不该被消费——除非它带着证据进系统。tests 保护它没明显破坏；docs 让未来的人读得懂；lineage 说清谁会被影响；artifacts 让验收可复核。这四件凑齐，才叫一次交付。

KEY THESIS · 我用了十几年的判断

负责的口径必须带证据——四件凑齐才叫交付

做事可以快，但这四件不能省；少一件，事故就以那件的形状出现。

tests

证明"明显破坏的事情没发生"。

not_null / unique / accepted_values + 自定义 SQL + unit test。

docs

让"未来的消费者"读得懂。

grain + 字段含义 + 时间语义 + 消费者清单。

lineage

让变更影响"可定位"。

manifest.json + DAG = 谁依赖谁。

artifacts

让交付"可验收"。

manifest / catalog / run_results / sources 四份。

TESTING · dbt 的四层测试

Generic / Singular / Unit / Source Freshness 各保护一类东西

不是写得越多越好。最常见反模式：只挂 not_null 凑覆盖率——这是工程的"表演性安全"，事故还是会来。

类别	怎么写	保护什么	Week05 示例
Generic · 内置通用	在 schema.yml 字段下挂 not_null / unique / accepted_values / relationships	主键、关键字段、枚举漂移、外键完整	ticket_id 唯一 / metric_name 在白名单
Singular · 自定义 SQL	在 tests/ 写 SELECT，返回任何一行 = 失败	复杂跨字段断言、安全边界	no_pii_columns_in_agent_tool_input_view ratio_metrics_between_0_and_1
Unit · dbt 1.8+	unit_tests 给输入打桩，验证 SQL 逻辑	复杂 case-when / 升级判定 / reopen 规则	"reopen 算不算"这种 SQL 逻辑回归
Source Freshness	sources.yml 配 loaded_at_field	"上游断流"工程层发现，而不是客户拍桌	ticket_fact 24h 内必须有新数据

真正保护口径的是"业务断言 + 自定义 + unit test"，不是 not_null 的数量。

LANDING · Week05 真实落地的测试

三个"业务断言", 是 metric_ready 的最低门槛

项目里这三段 SQL, 是我希望每个团队都"抄一份过去"的工程实践。

```
-- analytics/tests/no_pii_columns_in_agent_tool_input_view.sql
{{ config(tags=['week05']) }}
select column_name
from information_schema.columns
where table_schema = '{{ target.schema }}'
and table_name = 'agent_tool_input_view'
and column_name in (
  'contact_email','subject','body','body_preview','customer_id'
)
-- 任何一行 = 安全视图泄露了 PII 字段 → 红灯
```

no_pii_columns_*.sql

```
-- analytics/tests/ratio_metrics_between_0_and_1.sql
{{ config(tags=['week05']) }}
select *
from {{ ref('support_kpi_mart') }}
where metric_name in (
  'first_resolution_rate','escalation_rate','sla_breach_rate'
)
and (metric_value < 0 or metric_value > 1)
-- 越界几乎肯定是分子分母搞反
```

ratio_metrics_*.sql

第三个: metric_values_non_negative.sql —— count 类指标不能出现负值。这是口径完整性的底线。

DOCS · 写给"未来的消费者"

docs 不是负担，是消费者接口

我做评审时有一个第一性问题——"如果消费者把这个字段算错了，他能不能从文档里看出来？"答不出"能"，docs 就还不够。

```
# analytics/models/marts/schema.yml (节选)
models:
- name: support_kpi_mart
  description: "Controlled metric rows by date and safe business dimensions."
  columns:
    - name: metric_date
      description: "It comes from created_date in stg_tickets, not from updated_at..."
      tests: [not_null]
    - name: metric_name
      description: "Whitelisted metric identifier (see metric_registry_v1.yml)."
      tests:
        - not_null
        - accepted_values:
            values: ['ticket_count', 'p1_ticket_count', 'sla_breach_count', ...]
    - name: metric_value
      description: "Non-negative for count; 0..1 for ratio."
      tests: [not_null]
    - name: data_release_id
      description: "Required for reproducibility and audit."
      tests: [not_null]
```

schema.yml

必写 4 件事

- ① model description (一行 grain)
- ② 字段含义 + 时间语义
- ③ 边界 (不能怎么用)
- ④ 消费者清单

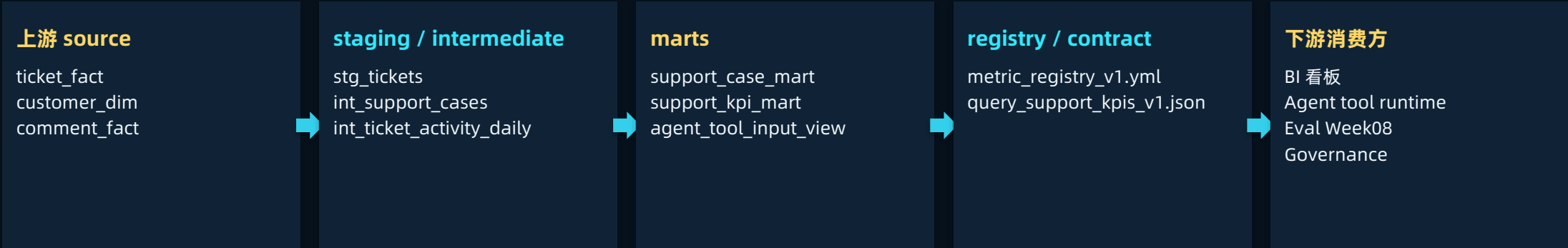
我的第一性问题

"消费者算错了，他能不能从文档里看出来？"
答不出"能"就还要补。

LINEAGE · 不是画图，是"变更影响入口"

看 DAG 永远要追问：改了它，谁会因为我而变？

我做评审时见过太多团队，把 lineage 截图发出去当交付——那不叫 lineage，那叫装饰。



你改了什么	下游可能受谁影响	必须同步更新的工件
stg_tickets · is_p1 定义	所有用 priority / p1_ticket_count 的 mart 与 dashboard	schema.yml / docs / registry / tool examples
int_support_cases · first_response_minutes	avg_first_response_minutes / SLA 看板 / Agent 工具	tests / docs / registry / impact note
support_kpi_mart · 加新指标	agent_tool_input_view / registry / tool contract	safe view + accepted_values test + registry validator

ARTIFACTS · dbt 给你的 4 份"事实证据"

manifest / catalog / run_results / sources —— 它们不是给课堂截图的

这 4 份 JSON 是 reviewer / 审计 / 复盘 / CI 的事实来源。把它们捡起来用，是工程级的最低门槛。

artifact	位置	记录了什么	在 Week05 怎么用
manifest.json	analytics/target/	所有节点、依赖、tests、资源元数据	抽 lineage 做影响分析；CI 比对前后差异
catalog.json	analytics/target/	仓库里真实的 schema、列、类型	回填到 dbt docs 网站；比对模型与仓库
run_results.json	analytics/target/	每次 run / build / test 的状态、耗时、错误	dbt_build_evidence.md 的事实来源
sources.json	analytics/target/	source freshness 检查结果	上游断流的工程预警

工程纪律：交付物里不允许只有"build 通过"截图——要带这 4 个 JSON 的关键摘录。

METRIC_READY GATE · 准入门槛 7 道闸

任一缺失 = HOLD。HOLD 是工程纪律，不是失败

这 7 道闸我在企业评审里反复用——任何一个团队跑不通 7 道闸，指标暂时不允许进 Agent 工具。

闸口	为什么必须有	没过该做什么
build pass	模型可以复现，不只是 stage 跑得通	先修模型；不要 patch 测试
tests pass	关键断言成立（口径合法、ratio 在 [0,1]）	修业务逻辑；不要 disable test
docs complete	消费者能读懂含义、边界、时间字段	HOLD，补字段 description
lineage clear	能定位变更影响范围	HOLD，补 impact note
registry entry	runtime 有事实来源（指标白名单 + 角色）	补 metric_registry_v1.yml 条目
safe view available	工具拿到的字段是安全的（无 PII）	修 safe view + 加 no_pii test
audit fields ready	出错能复盘（audit_id / release_id）	补输出契约 + 审计字段

TEMPLATE · dbt_build_evidence.md

交付物的标准格式：reviewer 3 分钟能复核

我反复跟团队讲：交付物里的每一行，都要让别人能复核。能复核 = 工程；不能复核 = 表演。

Week05 dbt Build Evidence

dbt_build_evidence.md

```
| Check | Command | Status |
|---|---|---|
| dbt debug | `cd analytics && DBT_PROFILES_DIR=. dbt debug` | passed: all checks |
| dbt build | `dbt build --select tag:week05` | passed: 39/39 |
| dbt docs | `dbt docs generate` | passed: catalog.json written |
| registry validator | `python analytics/scripts/validate_metric_registry.py --json` | passed: valid=true, metric_count=11 |
| KPI tool positive | `python -m app.kpi_query --example valid` | passed: allowed=true, row_count=16 |
| KPI tool negative | `python -m app.kpi_query --example bad_metric` | denial_code=METRIC_DENIED |
| Tool API endpoint | `POST /api/v1/tools/query_support_kpis` | passed: row_count=5 |
| Regression subset | `pytest tests/contract tests/integration -q` | passed: 53 passed |
```

TEMPLATE · lineage-impact-notes.md

变更影响：不要凭感觉，按模板写

这张模板让"你想到了什么、决定了怎么处理"变成显式记录——下次复盘的最值钱物料。

```
## Change
- changed_model: int_support_cases
- changed_field: first_response_minutes
- changed_logic: 只算 author_role=customer 的第一条评论；不再把 internal note 计入
- reason: 业务侧反馈 internal note 不应计入首响

## Upstream
- source: ticket_comment_fact
- staging: stg_ticket_comments （已暴露 author_role）

## Downstream impact
- marts: support_case_mart.first_response_minutes 含义变
- metrics: avg_first_response_minutes 历史回溯值会改 8-12%
- dashboards: 客服 SLA 看板
- tools: query_support_kpis_v1.avg_first_response_minutes 历史可比性下降
- evals: Week08 first-response 检索评测样本要 release_id 重新锚定

## Required updates
- docs: schema.yml.first_response_minutes 增加"仅 customer 首响"说明
- tests: unit test 增加 internal note 不计入的反例
- registry: business_definition_zh 增加"仅 customer"
- communications: 周二例会知会 Ops + BI； release_id 自 week05-v1.1.1 起生效
```

lineage-
impact-
notes.md

ANTI-PATTERNS · 我评审时直接拒签的 6 类

看起来交付了，实际上不可负责

这 6 类反模式，每一个都对应我赔过的钱。看到任一个 = HOLD。

① build screenshot only

只贴 dbt build 成功截图。
没有命令、没有选中模型、没有 test 数量。

② empty docs

schema.yml 只有字段名。
没有 description、没有时间语义说明。

③ DAG screenshot only

只贴漂亮 lineage 图。
没有"改了 X 谁会影响"的影响清单。

④ tests 全是 not_null

堆 not_null 凑数。
没有 ratio 边界、PII 测试、业务 case 断言。

⑤ registry stale

mart 改了，registry 没同步。
Agent 用旧白名单"成功"地拿到错数据。

⑥ safe view 无 PII test

view 没配 PII 边界测试。
下次新人忘了配置就泄露了。

CI/CD + SELF-CHECK

把证据链放进 CI；离开前过 6 个自检题

证据链如果只"这次做了"，下次还会漂；放进 CI 才是真正工程化。

① compile

dbt parse + dbt compile
语法 / Jinja / 依赖完整



② build

dbt build --select state:modified+
只跑改动 + 下游



③ docs + artifacts

dbt docs generate
上传 manifest/catalog/run_results



④ registry + tool

validate_metric_registry.py
pytest tests/contract tests/integration

① 我能解释为什么 build pass $\neq$ metric ready

② 我会写 3 种以上的 dbt 测试

③ 我知道 schema.yml 必写的 4 件事

④ 我能用 lineage 回答"改了 X 谁受影响"

⑤ 我能说出 manifest / catalog / run_results /
sources 各自的用途

⑥ 我知道 metric_ready 7 道闸

RECAP & NEXT

这节课你应该带走的 7 个判断

本课最重要的判断

- build pass $\neq$ metric ready; 要带证据链 tests / docs / lineage / artifacts。
- tests 是口径断言: generic / singular / unit / freshness 四层都不能少。
- docs 是写给未来消费者的接口; grain + 字段含义 + 时间语义 + 消费者。
- lineage 不是装饰图, 是"改了 X 谁会受影响"的工程入口。
- manifest / catalog / run_results / sources 是 4 份验收证据, 不要只截图。
- metric_ready 7 道闸: 任一缺失 = HOLD; HOLD 比事故便宜十倍。
- 下一讲: 把"能负责的口径"立成"语义契约"——本地 registry。

下一步

Lesson 04 会站在架构师角度判断: "要不要上 dbt Cloud / Cube / Looker; 什么时候上; 怎么过渡。"

你将带走: "先把可迁移的语义内核立住、再谈平台"的现实主义。

SEMANTIC LAYER · 给真实架构师的判断

4. 语义层的现实主义

先把内核立住，再谈平台外延。这是我跟所有客户讲的第一句话。

这节课不卖任何 SaaS——给你的是判断："要不要上、什么时候上、怎么过渡"。

本课交付

- 能讲清 metric / measure / dimension / entity / semantic graph 五个对象
- 一份决策记录：本周为什么走 local registry，而不是 dbt SL / Cube / Looker
- 在 OmniSupport 项目里跑通 `metric\_registry\_v1.yml` 的 v1.1 实验扩展

开场判断

事故视角

工程入口

行业信号

Recap/Next

Week05 学习链

01

Metrics

把业务口径变成工程接口

02

dbt Layers

sources / staging / intermediate / marts 的消费边界

03

Evidence

tests / docs / lineage 证明口径能负责

04

Semantics

MetricFlow、Semantic Models 与本地 registry 边界

05

Tool Guard

不让 Agent 裸写 SQL：受控指标查询工具

WAR STORY · 一个 60 万美元/年的真实判断

2025 某客户：上 dbt Cloud 的 Semantic Layer，花了一年多才发现"没准备好"

这件事让我后来推动"先本地、再平台"的过渡路径——不是反平台，是反"跳过准备就买票"。

事故现场

客户花 60 万美元/年订阅 dbt Cloud Semantic Layer。定义了 300 个指标，准备给 BI、CRM、Agent 共享。

一年后我去做架构盘点：BI 实际用了 78 个，Agent 用了 31 个。剩下 191 个"指标"——有定义、有 owner、没人用，且半数说不清 grain。

真正坏点不在平台

而在"组织还没准备好"——他们以为买了 SaaS 就有了语义层，但 metric_name 还没统一、grain 还没人能讲清。当年我给的判断：先把本地 registry 做扎实，验证 6 个月，再决定要不要上托管。同年他们退订到 Team 版。

我的判断不是反平台——是反"跳过准备就买票"。语义层的核心从来不是"用哪家"，是"你公司里同名指标能不能不变意思"。这个内核立不住，再贵的平台也救不了。

LANDSCAPE · 2025-2026 语义层市场

把市面上 6 类工具一次性拉开——你听到的"语义层"到底指什么

这张表上手就解决 80% 的"语义层 = 啥"混乱。

类型	代表	本质	与 Week05 的关系
仓内 SQL 工程化	dbt Core / SQLMesh	把 SQL 编译成可负责的模型	Week05 底座，必选
dbt SL (managed)	dbt Cloud Semantic Layer	托管 metric API: GraphQL / SQL / Python	不强依赖；可作未来迁移目标
Open MetricFlow	Apache 2.0 开源	dbt SL 的查询引擎	本周作为参考；不要求生产化
通用语义层平台	Cube / AtScale / GoodData	独立部署服务；含 cache / ACL / API	已有它的公司可平迁；不强求
BI 内置语义	LookML / Power BI 模型	绑定 BI 工具；Gemini 进入中	与 Agent 共享口径困难
OSI v1.0	2026.01 Apache 2.0	跨厂商语义层交换标准	让 registry 可跨平台迁移

BOUNDARY · 核心 vs 外延

你必须先立的，是核心；外延等核心稳了再谈

把核心立住，迁到任何平台都是"换前端"；核心没立，平台救不了你。

Semantic Core

metric_name
grain
dimensions
filters
roles + window

Local Registry

metric_registry_v1.yml
版本号 + owner
allowed_dimensions
sensitivity

Tool Contract

query_support_kpis_v1
JSON Schema strict
输入收紧

Semantic Model Draft

entity / measure
准 MetricFlow 形态
future mapping

Managed SL / APIs

dbt SL / Cube / OSI
GraphQL / MCP
optional extension

我的现实主义判断

Week05 只做前两件：Semantic Core + Local Registry。第三件 Tool Contract 下一节做完。

后两件（Semantic Model Draft + Managed SL）是迁移目标，不是本周硬要求。

本地可跑、可解释、可审计、可迁移——这条路径对所有公司都通用。

KEY THESIS

同一个词在不同系统里，不要变意思

语义层的核心不是平台名，是"事实来源"。这一句话立住，剩下的就是工程实施。

metric name

名称唯一，不复用歧义词。

ticket_count 这种名字本身有歧义；我宁可叫 ticket_created_count，让名字自己说话。

grain

一行 = 哪个粒度，必须可见。

Week05 标准：metric × date × dimensions。

dimensions / filters

可切维度与过滤白名单。

不允许"任意 where"。

role / time window

谁能查，能查多久（max_window_days）。

本质是访问控制 + 成本控制。

TERMS · 把语义层拆成 5 个工程对象

metric / measure / dimension / entity / semantic graph 不要混

混了你就读不懂 MetricFlow / dbt SL / Cube 的文档；也设计不出合格 registry。

术语	它是什么	它不是什么	OmniSupport 例子
semantic model	实体 + 维度 + 度量的语义结构	不是 dashboard，也不是单条 SQL	support_case / ticket_activity
entity	业务里"能被连接的对象"的主键	不是所有字段	case_id / customer_id / org_id
dimension	用来切分/过滤的属性	不是任意 where 条件	product_line / priority / category
measure	可聚合的基础值	不是最终业务指标	ticket_count / sla_breach_count
metric	面向业务消费的最终指标	不是 SQL 片段	p1_ticket_count / sla_breach_rate
semantic graph	语义对象之间的关系	不是漂亮图	case ◀ ▶ customer ◀ ▶ activity

REALITY CHECK · 四方案对比

挑方案的标准不是"先进"，是"五维都过得了"

本地可跑 / 语义一致性 / 工具边界 / 迁移性 / 运维成本——五个维度一起评。

方案	本地可跑	语义一致性	工具边界	迁移性	运维成本
dashboard SQL only	高	低	低	低	低
dbt marts only	高	中	中	中	中
local metric registry · 本周	高	高	高	高	中
Managed SL / MetricFlow	看部署	高	高	中（看 OSI）	高

本周选 local registry：先把"可迁移的语义内核"立住；将来要上 dbt SL / Cube，再做 mapping。

LANDING · registry 不是普通 YAML

metric_registry_v1.yml 是 runtime contract

Tool runtime 会按这份 YAML 校验每一次请求——它就是 Agent 与口径之间的"合法清单"。

```
# analytics/metric_registry_v1.yml (节选)
version: 1
registry_id: week05_support_metrics_v1
registry_version: 1.1.0
owner: analytics_engineering

source_model: support_kpi_mart
safe_view: agent_tool_input_view
time_dimension: metric_date
measure_column: metric_value
max_window_days: 31

allowed_dimensions: [product_line, priority, org_id, category]
allowed_filters: [product_line, priority, org_id, category, data_release_id]
allowed_roles: [support_ops, instructor, admin]

metrics:
- name: p1_ticket_count
  label: "P1 Ticket Count"
  business_name_zh: "P1 工单量"
  business_definition_zh: "priority 属于 p1 或 p1_critical 的工单数量。"
  owner: support_ops
  metric_type: count
  formula: "sum(case when is_p1 then 1 else 0 end)"
  aggregation: sum
  unit: tickets
  sensitivity: low
  definition_status: production
  version: 1.1.0
  allowed_roles: [support_ops, instructor, admin]
  quality_tests: [not_null, non_negative]
```

metric_registr
y_v1.yml

FIELDS · 每个字段都是 runtime 决策点

少一个字段，runtime 就缺一份判断

不是为了"完整"，是为了让 runtime 能在每一步问"我该放行吗"。

字段	回答的问题	缺失后果
metric_name	请求哪个指标	unknown metric 漂移
owner	出问题谁负责	事故无人认领
source_model / safe_view	从哪个 mart / 哪个安全视图	下游猜表 / 越权读 PII
grain / time_dimension	一行是什么 / 时间字段	时间错；重复 / 漏计
allowed_dimensions / filters	能按什么切 / 过滤	维度越权；任意 where
allowed_roles	哪些角色能查	越权访问
max_window_days	能查多久	成本爆 / 滥用
sensitivity / definition_status	敏感级 / 是不是实验代理	高敏感指标无显式确认
quality_tests	哪些测试在保护它	不可验收

v1.1 EXPERIMENTAL EXTENSION

从"可查询"推进到"可解释、可审计、可测试"

这是真实生产里语义层升级的典型路径——加字段，不改字段；旧调用全兼容。

新增指标 (5 个)

avg_first_response_minutes
avg_handle_time_minutes
first_resolution_rate  experimental
escalation_rate
sla_breach_rate

tool contract 新增输入

trace_id (端到端追踪)
purpose (声明用途)
actor_org_ids (组织范围)
include_experimental_metrics (实验指标确认)

registry 新增字段

owner / unit / metric_type / formula
sensitivity / definition_status
version / quality_tests

runtime 新增策略

audit_id / policy_applied / data_freshness
org_scope_filter
experimental_metric_ack

experimental_proxy · 工程师的"诚实"

first_resolution_rate 当前用「已解决且未升级 / 已解决」作为代理；真实生产需接入 reopen 事件后重定义。registry 标 experimental_proxy，runtime 强制 include_experimental_metrics=true 才放行。

HOLD IS NORMAL · 不开放也是工程能力

为什么不把所有指标立刻开放

我做架构这么多年最值钱的判断之一，是"什么时候说不"。

未通过关键 tests

ratio 范围 / not_null / unit test 没过。
开放它 = 高自信地错。

维度未授权

dimension 可能暴露敏感边界（如 assignee_id 涉及员工绩效）。
legal/security 没签字前不开。

窗口太大

max_window_days 还没定，怕扫表。
先用 31 天试运行。

PII / raw body 风险

safe view 没守住 或 no_pii test 没加。
直接 hold 工具入口。

grain 不稳定

一行还说不清；upstream 还有漂移。
等 mart 稳定后再开。

owner 不明确

没有人能解释这个指标。
找到 owner 再上线。

MIGRATION · 你的过渡路径

本地 registry → 正式 semantic model 的五步迁移

不是反平台，是把“可迁移的语义内核”先立住——避免被任何 SaaS 绑死。

① local registry

本周的事实来源
本地 YAML + validator

② semantic model draft

写成 MetricFlow YAML
entity / measure / metric

③ MetricFlow validate

本地 CLI 校验语义
回归测试

④ managed SL / OSI

dbt Cloud / Cube
或按 OSI 标准导出

⑤ BI/Agent/Eval 共享

下游统一消费
同一口径

我的迁移三原则

- ① 任何阶段都不要扔掉五个核心字段：metric_name / grain / allowed_dimensions / allowed_filters / allowed_roles + max_window_days。这就是 OSI 标准想统一的最低集合。
- ② 过渡期保持 registry 与 semantic model 双写一段时间——避免一刀切失败。
- ③ 监控三件事：迁移后口径数字一致性（指标级 diff）、下游延迟变化、SaaS 成本曲线。

OSI + SELF-CHECK

OSI v1.0: 你的 registry 字段已经 90% 与它对齐

2026 起绕不开的标准。这一页讲它是什么、对你意味着什么、自检一下。

OSI 是什么

Open Semantic Interchange · 跨厂商语义交换标准。

2026.01 v1.0 / Apache 2.0。

dbt Labs、Cube、Atlan、Snowflake 等签了。

它定义了 4 件事

- ① 指标的形状 (name / formula / unit)
- ② 维度与实体关系
- ③ 访问策略 (allowed_roles / allowed_filters)
- ④ 与 AI 工具的交互 (含 MCP 兼容)

① 我能说出语义层 5 个工程对象

② 我知道本地 registry 必填字段

③ 我能解释为什么 first_resolution_rate 是 experimental_proxy

④ 我能讲出"什么时候 HOLD"

⑤ 我知道五步迁移路径

⑥ 我知道 OSI 是什么、对我意味着什么

RECAP & NEXT

这节课你应该带走的 7 个判断

本课最重要的判断

- 语义层不是平台名，是 metric / measure / dimension / entity / semantic graph 五个工程对象。
- 本周走 local registry，是"先立内核、再谈平台"的现实主义。
- 不是所有可计算字段都要进 registry；HOLD 是工程纪律。
- registry 是 runtime contract，每个字段都是决策点。
- experimental_proxy 必须显式确认；这是工程师的"诚实"。
- 迁移路径有五步，五个核心字段一直不要丢。
- OSI v1.0 让你今天的 registry 未来跨平台几乎无痛。

下一步

Lesson 05 会把 registry 交给 Agent——但不让它裸写 SQL。

你将带走："schema 限形状，registry 限语义，guards 限权限和成本，audit 留证据"的完整工程纪律。

TOOL CONTRACT · LLM 时代的工程师必答题

5. 不让 Agent 裸写 SQL

把自由度压进可测试、可授权、可审计的工具里。

这一节讲一件事：你和 LLM 之间，不能是"信任和祈祷"，必须是"契约和闸门"。

本课交付

- 一份 tool contract (JSON Schema strict + additionalProperties=false)
- 一份 runtime (参数化 SQL + 7 道闸 + 9 种 denial_code)
- 一份 audit payload (trace_id / release_id / policy_applied)
- 在 OmniSupport 项目里跑通 4 个正负例 + Tool API endpoint

开场判断

事故视角

工程入口

行业信号

Recap/Next

Week05 学习链

01

Metrics

把业务口径变成工程接口

02

dbt Layers

sources / staging / intermediate / marts 的消费边界

03

Evidence

tests / docs / lineage 证明口径能负责

04

Semantics

MetricFlow、Semantic Models 与本地 registry 边界

05

Tool Guard

不让 Agent 裸写 SQL：受控指标查询工具

WAR STORY · 2024 那次 PII 越权事故

某客户 Agent 上线 3 个月后，数据库日志里大量 customer_email

这件事让我后来对所有团队说：Agent 永远不能写 SQL。

事故现场

客服把客户邮箱复制到 Agent 提示词里："帮我查 xxx@example.com 最近的工单"。Agent 为了"更好回答"，把 customer_email 当过滤条件写进了一段 SQL。结果：数据库 query log 里出现大量 customer_email = ... 的明文查询；DBA 备份系统也跟着把这些查询固化下来；合规审查发现，这是 PII 跨系统的非授权暴露。

修复花了多久

11 周。其中 8 周是合规审查、DPO（数据保护官）评估、客户通知；3 周是工程修复。这一次事故的总成本（含赔偿、审计、PR、人力），是这套 Agent 整套工具治理预算的 6 倍。

事故修完那天，我跟客户 CTO 说了一句话："Agent 永远不能写 SQL。" 不是它能力不够——是它没法对结果负责。这就是这一节的全部命题。

KEY THESIS · 不是"安全壳", 是"另一种工具"

受控工具不帮 Agent 拼 SQL; 它从根上换了接口形状

你和 LLM 不在 SQL 层斗智斗勇——你直接把 SQL 拿走, 给它一个只能传"指标 + 维度 + 时间窗"的入口。

① schema validation

形状先合法
JSON Schema strict

② registry lookup

指标 / 维度 / 过滤
必须来自白名单

③ guards

角色 / 窗口 / 成本
PII / 组织范围

④ parameterized SQL

确定性查询
不允许 raw SQL

⑤ audit payload

可复现 / 可追责 / 可评测
audit_id + release_id

五道闸缺一不可

schema 限"形状"; registry 限"语义"; guards 限"权限与成本"; 参数化 SQL 限"确定性"; audit 限"可复盘"。

做这种工具的"工程感"在于: 你能讲清每一道闸防的是什么风险——它会反过来逼你思考"以前哪一类事故是因为缺这一道"。

RISK TABLE · 这 6 类风险对应 6 道 guard

能跑的 SQL 也可能是错的 SQL

下次你做任何 LLM 工具，拿这张表来对照——少防一类，事故就以那一类的形状出现。

风险	具体表现	工具 guard
口径漂移	同名指标 SQL 不同；月会三个数	metric registry + accepted_values
越权访问	查到 customer_email / 评论正文 / 未授权维度	role policy + safe view
注入 / 非法过滤	Agent 接受任意 where；prompt 走漏 quoting	filter whitelist + 参数化 SQL
成本失控	365 天大窗口 / 全表扫 / limit 失控	max_window_days + limit cap
审计缺失	不知道谁查了什么；出问题无法复盘	audit payload + trace_id + release_id
不可复现	SQL 每次生成不同；答案漂	参数化 SQL + idempotency_key_fields

LLM PRIMER · 给零基础同学

Function Calling × Structured Outputs: Agent 调工具的两块基石

抽象的"Agent 调工具", 拆成 LLM 工程里两件明确支持的事——你才知道安全边界从哪儿来。

Function Calling 是什么

你告诉模型"我有这些工具, 每个输入是这样的 JSON Schema"。
模型输出"我要调用某工具 + 一段符合 schema 的 JSON"。
应用层再去执行真正的函数。

Structured Outputs (strict)

2024.08 OpenAI 推出。strict: true 后, 输出严格符合 JSON Schema。
4o-2024-08-06 实测 100%。
要求两件: additionalProperties: false + required 全列。

Week05 怎么用

把 query_support_kpis_v1.json 的 input_schema 直接交给 Function Calling。
模型生成的 payload 100% 形状合法; 剩下交给 runtime guards。

为什么 strict 还不够

strict 只保证"形状合法"。
它不知道指标真不真实、角色能不能查、31 天是不是太长。
这些必须 runtime 兜底。

TOOL DEFINITION · 它"不是"什么很关键

query_support_kpis_v1 是受控指标查询接口，不是 raw SQL 安全壳

它有意"不能"做很多事——这是它的工程价值，不是它的限制。

它是	它不是
registry-driven: 从 metric_registry_v1.yml 读真相	不是 NL2SQL; 不接受自然语言
parameterized: 只生成参数化 SQL	不是任意选表选字段; 不能 select customer_email
structured + audit: 返回结构化 rows + audit 字段	不是 raw SQL 安全壳; 不接受 raw_sql 参数
idempotent: 相同输入 → 相同结果	不是完整 Week10 治理; HITL / 配额还在后面接入

工具的价值在于"拒绝边界"，不在"灵活性"。

INPUT SCHEMA · 收紧到极致

additionalProperties: false 是 schema 真正的"安全锁"

这一行让"raw_sql"这种字段彻底进不来。即使 LLM 自作聪明加上, schema validate 立刻失败。

```
{
  "name": "query_support_kpis_v1",
  "version": "v1.1",
  "input_schema": {
    "type": "object",
    "required": ["actor_role", "metrics", "date_from", "date_to"],
    "properties": {
      "actor_role": { "type": "string" },
      "actor_id": { "type": "string" },
      "trace_id": { "type": "string" },
      "purpose": { "type": "string",
        "enum": ["classroom_demo", "support_ops_analysis",
          "incident_review", "regression_eval"],
        "default": "classroom_demo" },
      "actor_org_ids": { "type": "array", "maxItems": 50,
        "items": { "type": "string",
          "pattern": "^[A-Za-z0-9_-]+ $" } },
      "include_experimental_metrics": { "type": "boolean", "default": false },
      "metrics": { "type": "array", "minItems": 1, "maxItems": 11,
        "items": { "type": "string",
          "pattern": "^[a-z][a-z0-9_]* $" } },
      "date_from": { "type": "string", "format": "date" },
      "date_to": { "type": "string", "format": "date" },
      "dimensions": { "type": "array", "maxItems": 4 },
      "filters": { "type": "object" },
      "limit": { "type": "integer", "minimum": 1, "maximum": 500, "default": 100 }
    },
    "additionalProperties": false
  }
}
```

query_support_kpis_v1.json

RUNTIME · 6 步确定性查询

kpi_query.py 实际做的事 —— 每一步都不可绕过

整段不到 400 行，但你可以拿来当任何 LLM 工具的脚手架。

① load

load schema
load registry

② validate shape

jsonschema.validate
strict
→
SCHEMA_VALIDATION_FAILED

③ check policies

ROLE / METRIC /
DIMENSION
FILTER / WINDOW /
ORG_SCOPE
EXPERIMENTAL 七道闸

④ build SQL

参数化 placeholder
select from
analytics.safe_view

⑤ execute

asyncpg.fetch
→ DB_UNAVAILABLE 兜底

⑥ assemble

rows + audit +
data_freshness
policy_applied 标记本次
策略

```
def _build_query(payload, registry):
    where = ["metric_name = any(${1::text[]})",
            "metric_date between ${2::date} and ${3::date}"]
    params = [payload["metrics"], _parse_date(payload["date_from"]),
              _parse_date(payload["date_to"])]
    for field, value in payload.get("filters", {}).items():
        params.append([str(v) for v in (value if isinstance(value, list) else [value])])
        where.append(f"{field} = any(${len(params)}::text[])")
    if payload.get("actor_role") != "admin" and payload.get("actor_org_ids"):
        params.append([str(v) for v in payload["actor_org_ids"]])
        where.append(f"org_id = any(${len(params)}::text[])") # 行级权限
    return f"select ... from analytics.{registry.safe_view} where {' and '.join(where)} ...", params
```

kpi_query.py

_build_query

v1.1 ENHANCEMENTS · "加字段不破"

v1.1 在 v1 上只加 4 个字段——这是工具版本演进的标准姿势

你以后做任何工具升级，都该是这个模式：加字段保兼容，不改既有契约。

trace_id

端到端追踪 ID。

从 Agent → 工具 → DB → 评测的 5 跳，全用同一 trace_id 串起来。

purpose

声明用途：classroom_demo / support_ops_analysis / incident_review / regression_eval。

用于审计与策略匹配。

actor_org_ids

调用方的组织 ID 列表。

课堂版客户端传；生产版应由服务端身份解析后注入。

include_experimental_metrics

显式确认才允许 experimental_proxy 指标。

避免 LLM 在不知情时用代理口径"高自信地错"。

DENIAL CODES · 9 种拒绝码

拒绝码不是失败，是接口能力——它让 LLM"知道怎么改"

这一张表你可以印出来贴在工位旁。设计任何 LLM 工具时，这是必备清单。

code	触发条件	对 Agent 的提示
SCHEMA_VALIDATION_FAILED	输入不符合 JSON Schema	改 payload 形状
ROLE_DENIED	actor_role 不在 allowed_roles	换角色或申请权限
METRIC_DENIED	指标未注册 或 角色不允许	换指标 / 申请白名单
DIMENSION_DENIED	dimension 不在 allowed_dimensions	改维度选择
FILTER_DENIED	filters 字段不在 allowed_filters	改过滤字段
WINDOW_TOO_LARGE	date_to - date_from > max_window_days	缩小窗口
ORG_SCOPE_REQUIRED	support_ops 没传 actor_org_ids	补 org scope
EXPERIMENTAL_METRIC_NOT_ACKNOWLEDGED	experimental 指标未确认	设 include_experimental_metrics=true
DB_UNAVAILABLE	底层 DB 不可用	稍后重试或降级

AUDIT PAYLOAD · 6 个维度的"保命字段"

没有 audit 的工具调用不应进入生产

出事后，这 6 个维度让你能回答："谁查了什么、什么版本、走了哪些策略、得到什么结果"。

Who · 调用方

actor_id / actor_role / trace_id
purpose

What · 请求内容

metrics / dimensions / filters
date_from / date_to / actor_org_ids

Registry · 语义版本

registry_id / registry_version
safe_view

Data · 数据快照

release_id / data_release_id
generated_at_max

Outcome · 结果

row_count / denial_code
query_fingerprint (输入指纹)

Policy · 走的策略

policy_applied 列表: tool_contract / metric_registry /
safe_view / parameterized_sql / role_policy /
window_policy / org_scope_filter ...

LANDING · 项目 repo 路径全景

这套工程在 OmniSupport 里的物理位置

方法论 → 项目落地的对照表。你以后做客服 / 销售 / 风控的指标工具，路径几乎一样。

课堂概念	项目文件 / 路径
Tool contract	contracts/tools/tools/query_support_kpis_v1.json
Tool runtime	services/tool_api/app/kpi_query.py
Tool registry loader	services/tool_api/app/metric_registry.py
FastAPI 路由	services/tool_api/app/routers/kpis.py
Metric registry	analytics/metric_registry_v1.yml
Registry validator	analytics/scripts/validate_metric_registry.py
Safe view	analytics/models/marts/agent_tool_input_view.sql
自定义 PII 测试	analytics/tests/no_pii_columns_in_agent_tool_input_view.sql
Tool 范例 / 审计说明	reports/week05/query_tool_examples.md reports/week05/query_tool_audit_notes.md
Build 证据	reports/week05/dbt_build_evidence.md

WEEK05 CLOSING + SELF-CHECK

BI / Agent / Eval / 治理可以共享一套口径

Week05 的终点：业务口径升级成 BI / Agent / Eval / 治理共同承认的工程接口。

01 Metrics

metric interface

02 dbt Layers

consumption boundary

03 Evidence

tests / docs / lineage

04 Semantics

local registry

05 Tool Guard

controlled query

① 我能讲清"Agent 裸写 SQL"的 6 类风险

② 我能解释 schema strict 为什么不够

③ 我会用 9 种 denial_code

④ 我能说出 audit 6 个维度

⑤ 我能把整套方法论落到我自己公司的指标包

⑥ 我知道 Week06 / 08 / 10+ 怎么消费 Week05

RECAP & NEXT

Week05 收官：把指标包变成受控工程接口

本课最重要的判断

- Agent 可以问指标，但不能裸写 SQL ——这是 LLM 时代的工程师必答题。
- tool contract 不是提示词，是工程边界；JSON Schema strict + additionalProperties=false 是关键。
- registry 是 runtime 的事实来源；safe view 是数据边界；参数化 SQL 是安全底线。
- structured result 是审计和评测基础；audit payload 是生产复盘入口。
- v1.1 "加字段不破"是工具版本演进的标准姿势——你以后做任何工具都这么干。
- 9 种 denial_code 让 LLM "知道怎么改"——这是把 LLM 当作"可学习客户端"对待。
- Week05 指标包会被 Week06 / Week08 / Week10+ 继续消费——它不是终点，是地基。

下一步

完成 Week05 后，你应该能交付一份可负责的指标包 v1：dbt models + tests/docs/lineage + local registry + controlled query tool + audit evidence。

下一步是实验课——我带你把这套方法论，落到 docker compose、dbt build、registry validator、kpi tool 正负例、Tool API endpoint、回归测试这一系列具体操作。